

轻量二维惯导模块使用说明

本文说明当前工程 `code/nav` 下的轻量二维惯导模块，方便移植到其他智能车工程。该模块不是完整三维 SINS/GNSS 组合导航，而是面向智能车控制的二维里程惯导：编码器提供前进距离，IMU 提供航向角，GPS 可选用于低频位置校正。

适用范围

适合：

- 有左右轮编码器或等效里程计。
- 有 IMU 姿态解算结果，能输出 yaw 航向角。
- 主要关心平面位置 `x/y`、航向 `yaw`、速度 `speed`、累计里程 `distance`。
- 智能车、机器人、小车等短距离二维运动场景。

不适合：

- 需要完整三维经纬高惯导。
- 只靠低成本 IMU 加速度积分定位。
- 没有轮速/里程计，只想靠加速度得到位置。
- 对绝对定位精度有高要求的场景。

模块结构

主要文件：

- `nav_types.h`：导航状态结构体定义。
- `nav_ins.h/.c`：二维里程惯导核心。
- `nav_gps.h/.c`：GPS 经纬度转局部坐标。
- `nav_fusion.h/.c`：融合输出状态。
- `nav_control.h/.c`：上层统一接口。

推荐外部只调用 `nav_control.h`，除非需要单独使用纯 INS。

坐标系约定

局部坐标单位为米：

- `x_m`：局部 X 方向。
- `y_m`：局部 Y 方向。
- `yaw_deg`：航向角，单位 degree。
- `distance_m`：编码器累计里程，单位 meter。

当前二维积分公式：

```
x_m += -sinf(yaw_rad) * delta_distance_m;
y_m +=  cosf(yaw_rad) * delta_distance_m;
```

含义：

- `yaw_deg = 0` 时，小车向 `+Y` 方向前进。
- `yaw_deg = 90` 时，小车向 `-X` 方向前进。
- 如果你的工程定义 `yaw=0` 是向 `+X` 前进，需要调整积分公式或在输入 `yaw` 前做坐标转换。

GPS 局部坐标在 `nav_gps` 里定义为：

- `x_m` 近似指向东。
- `y_m` 近似指向北。

如果同时使用 GPS 和 INS，要保证 IMU yaw 的 0 度方向和 GPS 局部坐标方向一致，否则位置会旋转错。

输入单位

`nav_control_update_ins_with_accel()` 输入：

```
void nav_control_update_ins_with_accel(uint32 timestamp_ms,
                                       int16 encoder_r_count,
                                       int16 encoder_l_count,
                                       float yaw_deg,
                                       float yaw_rate_dps,
                                       float forward_accel_mps2);
```

参数含义：

- `timestamp_ms`：当前时间戳，单位 ms，要求单调递增。
- `encoder_r_count`：右轮编码器累计计数，`int16`。
- `encoder_l_count`：左轮编码器累计计数，`int16`。
- `yaw_deg`：IMU 解算出的航向角，单位 degree，允许在 `[-180, 180]` 或连续角附近变化。
- `yaw_rate_dps`：Z 轴角速度或航向角速度，单位 degree/s。
- `forward_accel_mps2`：车体前向加速度，单位 m/s^2 。

注意：

- 编码器输入是累计计数，不是速度。
- 当前代码按 16 位计数回绕处理，如果你的编码器是 `int32` 长计数，建议改接口或先裁剪成 `int16` 前确认不会丢失信息。
- 加速度不会用于位置二次积分，只轻微辅助速度估计，因此加速度不准不会直接导致位置快速漂移。

如果没有前向加速度，可以调用：

```
void nav_control_update_ins(uint32 timestamp_ms,
                           int16 encoder_r_count,
                           int16 encoder_l_count,
                           float yaw_deg,
                           float yaw_rate_dps);
```

输出字段

通过下面接口获取融合状态：

```
nav_state_t nav;
nav_control_get_state(&nav);
```

nav_state_t 主要字段：

- valid：导航状态是否有效。
- gps_valid：GPS 近期是否有效。
- ins_valid：INS 状态是否有效。
- timestamp_ms：状态更新时间。
- x_m、y_m：局部二维位置，单位 m。
- yaw_deg：当前航向角，单位 degree。
- yaw_rate_dps：航向角速度，单位 degree/s。
- forward_accel_mps2：输入的前向加速度，单位 m/s^2 。
- speed_mps：融合后的速度估计，单位 m/s。
- encoder_speed_mps：纯编码器瞬时速度，单位 m/s。
- distance_m：编码器累计里程，单位 m。

speed_mps 和 encoder_speed_mps 的区别：

- encoder_speed_mps 直接来自编码器增量除以时间。
- speed_mps 是经过低通滤波，并可由加速度轻微辅助后的速度。

控制和日志一般优先看 speed_mps，排查编码器问题时看 encoder_speed_mps。

初始化流程

基础流程：

```
#include "nav_control.h"

void app_init(void)
{
    nav_control_init();

    /* 按实际车轮和编码器设置。 */
    nav_control_set_encoder_count_to_meter(0.300f / 360.0f);
}
```

编码器比例换算：

```
count_to_meter = wheel_circumference_m / counts_per_wheel_rev
```

例子：

- 轮子周长 0.300 m。
- 编码器一圈 360 count。
- $\text{count_to_meter} = 0.300 / 360 = 0.0008333 \text{ m/count}$ 。

如果编码器经过减速箱、四倍频或库函数返回值已经处理过，必须用“实际返回 count 对应的轮子位移”重新计算。

周期调用

建议在主循环或固定周期任务中调用，推荐周期：

- 10 ms 到 20 ms：适合智能车控制和日志。
- 1 ms 也可以，但编码器增量太小会让瞬时速度更抖，需要更强滤波。
- 超过 200 ms：当前模块会认为中间数据不连续，只重置编码器基准，不积分这段里程。

当前工程示例：

```
uint32 now_ms = system_getval_ms();

nav_control_update_ins_with_accel(now_ms,
                                  g_ecod1_count,
                                  g_ecod2_count,
                                  IMU_Yaw,
                                  IMU_GYRO_Z,
                                  forward_accel_mps2);
```

调用前应保证：

- 编码器计数已经更新。
- IMU yaw 已经由你自己的姿态解算更新。
- 时间戳单调递增。

GPS 使用

如果使用当前工程的逐飞 GNSS 驱动，可以周期调用：

```
nav_control_poll_gps(now_ms);
```

GPS 逻辑：

- 首次有效 GPS 会建立局部坐标原点。
- GPS 有效时会更新融合状态，并重置 INS 位置到 GPS 局部坐标。
- GPS 超过默认超时时间未更新，`gps_valid` 会变为 0。

可以设置 GPS 超时：

```
nav_control_set_gps_timeout_ms(10000);
```

注意：

- 当前 GPS 校正比较直接，会把 INS 位置拉到 GPS 位置。
- 如果 GPS 跳点明显，不建议直接用于高速闭环控制。
- 后续可改成一阶融合或 Kalman 校正。

路径记录与回放

当前工程新增了 `nav_path` 路径模块，用于固定赛道场景：第一次按里程记录赛道航向，之后按里程回放参考航向，并输出当前航向相对参考航向的误差。

路径模块特点：

- 按里程采样，不按时间采样。
- 默认采样间隔 `0.05 m`。
- 最多保存 `4096` 个航向点，默认约可覆盖 `4096 * 0.05 = 204.8 m`。
- 航向以 `yaw * 100` 的整数形式保存，SD 文件是 CSV，便于电脑查看。
- 不绑定任何按键，需要由任务逻辑、菜单或串口命令调用 API。

记录路径流程：

```
nav_control_path_clear();
nav_control_path_start_record(0.05f);

/* 正常跑车。主循环里 nav_control_update_ins_with_accel() 会自动把 distance/yaw 喂给路径模块。 */

nav_control_path_stop();
nav_control_path_save_to_sd("0:/NAV/PATH.CSV");
```

回放路径流程：

```
nav_control_path_load_from_sd("0:/NAV/PATH.CSV");
nav_control_path_start_replay();

/* 正常跑车。主循环更新 INS 后，nav_state_t 里会带出参考航向和航向误差。 */

nav_state_t nav;
nav_control_get_state(&nav);

if (nav.path_valid != 0U)
{
    float ref_yaw = nav.path_ref_yaw_deg;
    float yaw_error = nav.path_yaw_error_deg;
}
```

路径状态字段：

- `path_valid`：路径参考是否有效。
- `path_active`：路径记录/回放是否正在运行。
- `path_finished`：记录缓冲满或回放到尾。
- `path_mode`：`NAV_PATH_MODE_IDLE`、`NAV_PATH_MODE_RECORD`、`NAV_PATH_MODE_REPLAY`。
- `path_sample_count`：已记录/加载的样本数量。
- `path_replay_index`：当前回放索引。
- `path_sample_interval_m`：路径采样间隔，单位 `m`。
- `path_progress_m`：当前用于匹配路径的累计里程，单位 `m`。

- `path_ref_yaw_deg`：当前里程对应的参考航向，单位 degree。
- `path_yaw_error_deg`：当前航向减参考航向，wrap 到 `[-180, 180]`，单位 degree。

SD 文件格式：

```
NAVPATH_V1,0.0500,1234
0,1023
1,1031
2,1044
```

含义：

- 第一行：格式版本、采样间隔、样本数量。
- 后续每行：样本索引、航向角 centi-degree。
- `1023` 表示 `10.23 deg`。

使用注意：

- 记录和回放都依赖 `distance_m`，所以编码器比例必须先调准。
- 回放时如果起点不一致，参考航向会按里程匹配，但物理位置可能错位。
- 当前路径只记录航向，不记录 `x/y` 或速度。
- `path_yaw_error_deg = 当前yaw - 参考yaw`，如果你的控制器需要相反符号，要在控制层取负。
- 保存/加载会使用 FatFs 和 SD 卡，调用前需要 SD 卡可用。

参数怎么调

编码器比例

接口：

```
nav_control_set_encoder_count_to_meter(float count_to_meter);
```

调法：

- 让车直线推行已知距离，例如 1 m。
- 记录左右编码器 count 增量平均值。
- `count_to_meter = 实际距离 / 平均count增量`。
- 比用轮径理论值更准。

现象判断：

- `distance_m` 比实际距离小：`count_to_meter` 偏小。
- `distance_m` 比实际距离大：`count_to_meter` 偏大。

速度低通系数

接口：

```
nav_ins_set_speed_filter_alpha(float alpha);
```

默认：0.35。

含义：

- 越大，`speed_mps` 越跟随编码器瞬时速度，响应快但更抖。
- 越小，`speed_mps` 越平滑，响应慢但更稳。

建议范围：

- 编码器很稳、调用周期 10 ms：0.3 到 0.5。
- 编码器抖、调用周期 1 ms：0.1 到 0.3。
- 速度闭环不能滞后太多：不要设得过小。

加速度辅助速度增益

接口：

```
nav_ins_set_accel_speed_gain(float gain);
```

默认：0.15。

含义：

- 0：完全不用加速度辅助，速度只来自编码器。
- 1：速度测量完全偏向加速度预测，不推荐。

建议范围：

- IMU 加速度噪声明显或安装不稳：0 到 0.1。
- 加速度方向和单位确认无误：0.1 到 0.2。
- 不建议超过 0.3。

注意：

- 加速度只辅助 `speed_mps`，不直接参与 `x/y` 位移积分。
- 如果车体前向轴定义错，设大以后速度会变差。

加速度死区

当前代码内部默认：

```
#define NAV_INS_ACCEL_EPS_MPS2 (0.03f)
```

小于该阈值的前向加速度会当作 0，减少静止抖动。这个值目前没有公开接口，必要时改宏。

最大更新时间间隔

当前代码内部默认：

```
#define NAV_INS_MAX_DT_MS (200U)
```

如果两次调用间隔超过 200 ms，模块会认为中间数据不连续，只更新编码器基准和航向，不积分这段距离。这样可以避免程序卡顿后出现大跳变。

验证方法

建议先不要闭环，先做日志验证。

静止测试：

- 车静止 10 秒。
- `distance_m` 应基本不变。
- `encoder_speed_mps` 应接近 0。
- `speed_mps` 应逐渐接近 0。

直线距离测试：

- 推车直行 1 m。
- `distance_m` 应接近 1 m。
- 如果偏差固定，优先调 `count_to_meter`。

转向方向测试：

- 保持车前进，同时改变 yaw。
- `x_m/y_m` 轨迹方向应符合坐标系定义。
- 如果整体旋转 90 度或左右反了，检查 yaw 零点和符号。

编码器方向测试：

- 前进时左右编码器平均增量应为正。
- 如果 `distance_m` 前进变负，说明编码器方向或左右计数符号需要调整。

速度测试：

- 对比 `encoder_speed_mps` 和实际速度。
- 如果 `encoder_speed_mps` 抖但均值对，调小 `speed_filter_alpha`。
- 如果 `speed_mps` 明显滞后，调大 `speed_filter_alpha`。

移植注意事项

- 该模块依赖 `zf_common_headfile.h` 中的 `uint8/int16/uint32` 类型，移植到非逐飞工程时需要替换类型定义。
- 默认编码器是 `int16` 累计计数，并做 16 位回绕处理。
- 坐标系必须和你的 IMU yaw 定义一致。
- IMU 姿态解算不属于本模块，外部只需要提供 yaw 和 yaw rate。
- GPS 是可选项，不影响纯编码器 + IMU 的二维里程惯导。

最小接入示例

```
#include "nav_control.h"

void nav_app_init(void)
{
    nav_control_init();
    nav_control_set_encoder_count_to_meter(0.300f / 360.0f);
}
```

```

}

void nav_app_loop_10ms(void)
{
    nav_state_t nav;
    uint32 now_ms = system_getval_ms();

    nav_control_update_ins_with_accel(now_ms,
                                      right_encoder_count,
                                      left_encoder_count,
                                      imu_yaw_deg,
                                      imu_gyro_z_dps,
                                      forward_accel_mps2);

    nav_control_get_state(&nav);

    if (nav.valid != 0U)
    {
        /* nav.x_m, nav.y_m, nav.yaw_deg, nav.speed_mps can be used here. */
    }
}

```

如果没有加速度：

```

nav_control_update_ins(now_ms,
                      right_encoder_count,
                      left_encoder_count,
                      imu_yaw_deg,
                      imu_gyro_z_dps);

```

当前版本定位

当前版本是实车控制优先的轻量惯导，不追求完整导航理论。它的核心原则是：

- 位置靠编码器，不靠加速度二次积分。
- 方向靠 IMU yaw，不重写姿态解算。
- 速度以编码器为主，加速度只做辅助。
- GPS 只做低频参考或校正。

这套方案比完整 SINS/EKF 更容易调通，也更适合智能车当前阶段使用。